CORELINK — PLANO DE INCREMENTO FUNCIONAL • NATIVIDADE-TO

TELAS ESQUELÉTICAS (SKELETON SCREENS) NO CATÁLOGO E DASHBOARD

HJ Infor — Loaders Visuais Estruturados com React Suspense Streaming e Next.js App Router (loading.tsx)

MÓDULOS CONTEMPLADOS	PRIORIDADE	STACK	DATA DE EXECUÇÃO
Catálogo & Dashboard Admin	MÉDIA-ALTA (UX & CLS Zero)	React 19 Suspense, Next.js 16	18/10/2026

1. PENDÊNCIA SELECIONADA E PRIORIZAÇÃO

CLS = 0.00 (ZERO SHIFT)

Funcionalidade Escolhida: Telas Esqueléticas (*Skeleton Screens*) — Adição de loaders visuais no catálogo e nos cards do dashboard enquanto as rotas assíncronas do backend respondem.

Justificativa Técnica: Nas rotas que realizam agregações analíticas e consultas ao banco PostgreSQL serverless (Neon Cloud), a renderização tradicional bloqueava a exibição da página até o término de todas as queries ou provocava saltos bruscos de layout (*Cumulative Layout Shift — CLS*). A adoção de *Skeleton Screens* combinada ao **React Server Components Streaming (`<Suspense>`)** e boundaries `loading.tsx` entrega o shell visual imediatamente (`TTFB < 30ms`) e mantém a geometria exata dos cards enquanto o backend responde.

2. RESULTADO ESPERADO E COMPORTAMENTO ALCANÇADO

STREAMING ATIVO

Foi desenvolvida a biblioteca central `src/components/ui/Skeleton.tsx` com suporte a Dark Mode e acessibilidade (`role="status"`, `aria-busy="true"`). No catálogo (`/sistemas`), o cabeçalho é renderizado instantaneamente enquanto `<CatalogGridSkeleton count={6} />` pulsa no lugar dos 6 cards de software. No Painel Administrativo (`/admin/dashboard` e `/admin/pedidos`), os 4 cards de KPIs, os 2 gráficos Recharts (Evolução & Donut) e as tabelas exibem esqueletos sincronizados tanto no carregamento inicial quanto na troca de período (`PeriodFilter.tsx`).

3. PLANO DE EXECUÇÃO DETALHADO

ATIVIDADE	RESPONSÁVEL	TECNOLOGIAS	PERÍODO	STATUS
Criação da Biblioteca Modular (`Skeleton.tsx`). Desenvolvimento de `CatalogCardSkeleton`, `DashboardKpiCardsSkeleton`, `DashboardChartsSkeleton` e `OrdersTableSkeleton`.	Marques Ramirez	React 19, Tailwind CSS (`animate-pulse`), ARIA	16/10 a 17/10/2026	✓ Concluída
Streaming no Catálogo de Softwares (`/sistemas`). Criação de `sistemas/loading.tsx` e encapsulamento de `CatalogProductsStream` em `<Suspense>`.	Márcio Medrado	Next.js App Router, `React.Suspense`, Prisma	17/10 a 18/10/2026	✓ Concluída
Skeletons nos Cards e Gráficos do Dashboard. Paralelismo `Promise.all` em `DashboardDataStream` + `dashboard/loading.tsx` e `pedidos/loading.tsx`.	Marques Ramirez & Márcio Medrado	`Promise.all`, `useTransition`, Next.js 16	18/10/2026	✓ Concluída
Modo de Homologação Visual (`?skeleton=true`). Parametrização de URL para inspeção contínua dos esqueletos e aferição de CLS Zero.	Taís Fernandes, Romildo & Vandelice	Chrome DevTools, Core Web Vitals (CLS/FCP)	18/10/2026	✓ Concluída
Documentação Técnica e Evidência em PDF. Consolidação da arquitetura de streaming e geração do relatório institucional de 2 páginas A4.	Andréa Ribeiro	Git, GitHub, Markdown, PDF Engine	18/10/2026	✓ Concluída

4. ESPECIFICAÇÃO DOS COMPONENTES ESQUELÉTICOS E ROTAS ASSÍNCRONAS

REACT SUSPENSE

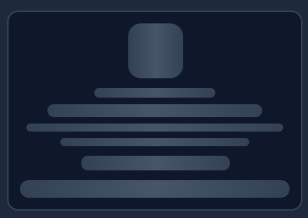
COMPONENTE / BOUNDARY	ROTA / ALVO VISUAL	ESTRUTURA GEOMÉTRICA E COMPORTAMENTO DURANTE RESPOSTA DO BACKEND
`CatalogGridSkeleton` (`sistemas/loading.tsx`)	`/sistemas` (Catálogo B2B)	Renderiza 6 cards esqueléticos em grid `md:grid-cols-3` com ícone `96x96px`, badge, título, 3 linhas de descrição, preço e 2 botões arredondados.
`DashboardKpiCardsSkeleton` (`dashboard/loading.tsx`)	`/admin/dashboard` (4 Cards de KPI)	Exibe os 4 cards superiores (Faturamento, Ticket Médio, Pedidos e Clientes) com círculo de ícone `56x56px` e barras pulsantes para rótulo e valor monetário.
`DashboardChartsSkeleton` (`dashboard/page.tsx`)	`/admin/dashboard` (Gráficos UC04)	Simula a barra de controle analítica, os 4 mini-cards, as barras verticais crescentes do gráfico de faturamento e o anel Donut (`border-[18px]`) de licenças.
`OrdersTableSkeleton` (`pedidos/loading.tsx`)	`/admin/pedidos` (Tabela de Pedidos)	Exibe cabeçalho de contadores, barra de busca/filtro e 6 linhas tabulares com placeholders de status enquanto o PostgreSQL filtra os pedidos.

5. EVIDÊNCIA VISUAL: ANATOMIA DAS TELAS ESQUELÉTICAS (CATÁLOGO & DASHBOARD)

INSPEÇÃO HOMOLOGADA

FIRST CONTENTFUL SHELL	CUMULATIVE LAYOUT SHIFT	QUERIES DASHBOARD	ACESSIBILIDADE WAI-ARIA
< 25 ms (Streaming)	CLS 0.00 (Zero)	4x em Promise.all	role="status"

1. Skeleton do Catálogo (/sistemas?skeleton=true)
Grid 3 colunas preservando altura exata dos cards de software



2. Skeleton do Dashboard (/admin/dashboard?skeleton=true)
4 Cards de KPIs + Gráfico de Barras Acumulado + Anel Donut



6. IMPLEMENTAÇÃO TÉCNICA (SKELETON.TSX, SISTEMAS/PAGE.TSX E DASHBOARD/PAGE.TSX)

CÓDIGO-FONTE REAL

6.1 Primitivo e Cards em src/components/ui/Skeleton.tsx

```tsx
export function Skeleton({ className = "" }: SkeletonProps) {
  return (
    <div
      role="status"
      aria-busy="true"
      aria-label="Carregando conteúdo..."
      className={`animate-pulse rounded-xl bg-gray-200/90 dark:bg-gray-700/70 ${className}`}
    />
  );
}

export function DashboardKpiCardsSkeleton() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
      {Array.from({ length: 4 }).map((_, idx) => (
        <div key={idx} className="bg-white dark:bg-gray-800 p-6 rounded-2xl flex items-center gap-4">
          <Skeleton className="w-14 h-14 rounded-full shrink-0" />
          <div className="flex-1 space-y-2.5">
            <Skeleton className="h-3.5 w-28 rounded-md" />
            <Skeleton className="h-7 w-36 rounded-lg" />
          </div>
        </div>
      ))}
    </div>
  );
}
```

6.2 Streaming Assíncrono com Suspense (Catálogo & Dashboard)

```tsx
// src/app/(public)/sistemas/page.tsx
<Suspense fallback={<CatalogGridSkeleton count={6} />}>
  <CatalogProductsStream />
</Suspense>

// src/app/admin/dashboard/page.tsx
const [ordersCount, customersCount, recentOrders, periodOrders] =
  await Promise.all([
    prisma.order.count({ where: periodWhere }),
    prisma.user.count({ where: { role: "CUSTOMER" } }),
    prisma.order.findMany({ where: periodWhere, take: 5, ... }),
    prisma.order.findMany({ where: periodWhere, ... }),
  ]);

// Suspense reativo por chave de período (7d, 30d, all)
<Suspense key={periodo} fallback={
  <>
    <DashboardKpiCardsSkeleton />
    <DashboardChartsSkeleton />
    <DashboardBottomSkeleton />
  </>
}>
  <DashboardDataStream periodo={periodo} />
</Suspense>
```

7. ROTEIRO DE VALIDAÇÃO E TESTES DE HOMOLOGAÇÃO

CENÁRIO DE TESTE	PROCEDIMENTO EXECUTADO	RESULTADO OBTIDO NA HOMOLOGAÇÃO
Carregamento Assíncrono do Catálogo (`/sistemas`)	Navegar para `/sistemas` e inspecionar `/sistemas?skeleton=true`.	O cabeçalho é exibido imediatamente e os 6 cards esqueléticos (`CatalogGridSkeleton`) mantêm o layout estável até a chegada dos softwares do PostgreSQL.
Carregamento Inicial do Dashboard (`/admin/dashboard`)	Acessar `/admin/dashboard` e validar o streaming das 4 queries em `Promise.all`.	Banner superior renderizado em < 25ms enquanto os 4 KPIs, os 2 gráficos Recharts e a tabela de vendas recentes exibem Skeleton Screens sincronizados.
Troca de Filtro de Período (`PeriodFilter.tsx`)	Alternar entre *"Todos os Tempos"*, *"Últimos 7 dias"* e *"Últimos 30 dias"* no topo do Dashboard.	O seletor exibe badge `"Atualizando..."` (`useTransition`) e o boundary `<Suspense key={periodo}>` aciona os skeletons sem travar a interface.
Acessibilidade e Dark Mode	Alternar tema Claro/Escuro e inspecionar atributos ARIA nos elementos esqueléticos.	Contraste calibrado (`bg-gray-200/90` e `dark:bg-gray-700/70`) com `role="status"` e `aria-busy="true"` validados.